

École nationale supérieure des Mines de Saint-Etienne

LaTeXProject
Rapport

TD 2 - Architecture des Processeurs

Élèves :
Samy TALEB
Ylies VEYRAT

Encadrants :
PONTIE Simon
GOUSSELOT Théophile

18 décembre 2024

Table des matières

1 Introduction et mise en contexte **3**

2 Intégration du banc de registres 32 bits **4**
2.1 Question 1 4

3 Ajout d'une instruction **5**
3.1 Question 2 5
3.2 Question 3 5
3.3 Question 4 5
3.4 Question 5 6
3.5 Question 6 7
3.6 Question 7 7

4 Ajout des types I et U **8**
4.1 Question 8 8
4.2 Question 9 9
4.3 Question 10 10

1 Introduction et mise en contexte

Tout au long de ce TD, nous élaborerons une microarchitecture monocycle. Cette microarchitecture se compose principalement de deux éléments :

1. Le data-path, où les divers composants de la microarchitecture tels que le compteur de programme, la mémoire d'instructions, le banc de registres, et l'unité arithmétique et logique sont connectés pour le traitement des données.

2. Le control-path, qui supervise le data-path et modifie sa configuration pour traiter les données conformément à l'instruction en cours d'exécution.

2 Intégration du banc de registres 32 bits

2.1 Question 1

Il faut instancier le banc de resgistre dans le data_path pour qu'il puisse echanger entre l'alu et la memoire.

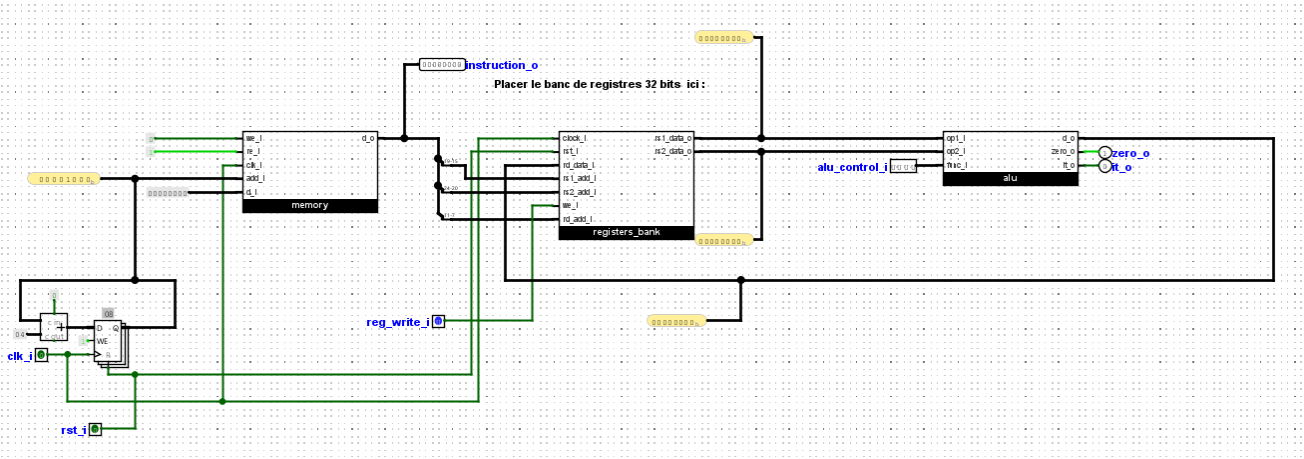


FIGURE 1 – Circuit comprenant le banc de registres

3 Ajout d'une instruction

3.1 Question 2

Dans le data-path, seules les instructions registre-registre sont supportées, comme indiqué par le fait que seul le bloc registers_bank est relié à l'ALU. Cela correspond à l'architecture monocycle mentionnée dans la réponse précédente. De plus, dans le control_path, on ne teste que l'égalité avec l'opcode des instructions de type R, ce qui confirme que seules les instructions de type registre sont prises en charge. En ce qui concerne la mémoire, elle est connectée à un registre qui est relié à un additionneur +4 qui est le Program Counter. Cela signifie que chaque instruction est codée sur 4 octets et le passage d'une instruction à une autre nécessite d'ajouter 4 à l'adresse précédente.

3.2 Question 3

Les instructions sont contenues dans la mémoire donc dans le bloc «memory».

3.3 Question 4

Pour exécuter les deux instructions fournies (001080B3 et 001080B3), nous devons positionner les valeurs initiales des composants comme suit :
- Pour le composant BL0 : Entrer B3 dans la première et la deuxième adresse.
- Pour le composant BL1 : Entrer 80 dans la première et la deuxième adresse.
- Pour le composant BL2 : Entrer 10 dans la première et la deuxième adresse.
- Pour le composant BL3 : Entrer 00 dans la première et la deuxième adresse.

Voici les schémas associés à chacunes des instructions citées précedemment :

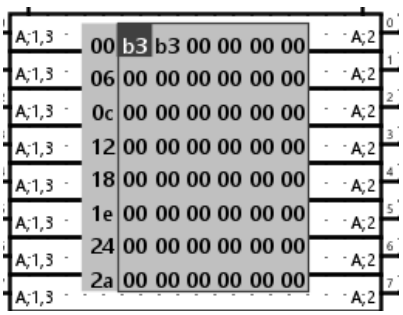


FIGURE 2 – Composant BL0

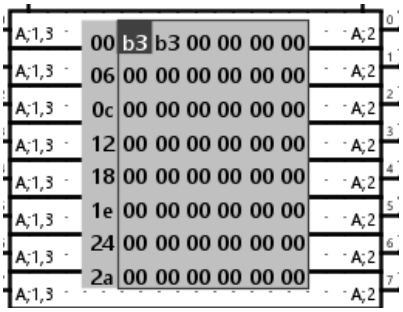


FIGURE 3 – Composant BL1

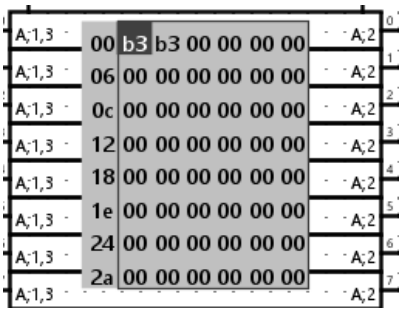


FIGURE 4 – Composant BL2

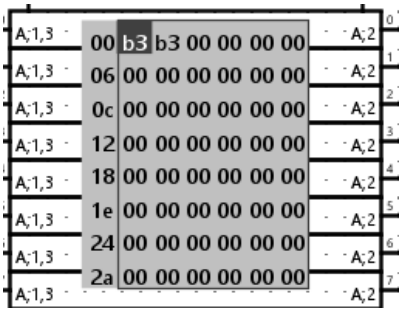


FIGURE 5 – Composant BL3

3.4 Question 5

On convertit les instructions hexadecimales en binaires pour y identifier l'Opcode, , le type, les registres :
0000000 00001 00001 000 00001 0110011
0000000 00001 00001 000 00001 0110011
En identifiant les instructions correspondantes, on obtient que l'on souhaite mettre dans R1 sa valeur multipliée par 4 (on multiplie la valeur de R1 par 2 puis encore par 2).

$$R1 = R1 * 2$$

$$R1 = R1 * 2$$

Au final,

$$R1 = R1 * 4 \tag{1}$$

3.5 Question 6

Après avoir instancié le registre R1 a 1 et après avoir initialisé les mémoires comme défini à la question précédente on obtient que :

$$R1 = 4.$$

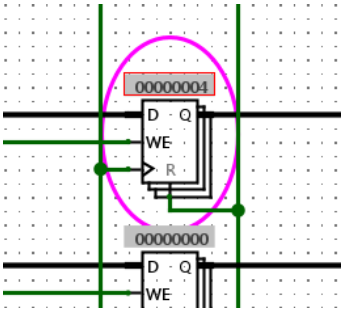


FIGURE 6 – Valeur obtenue pour R1

3.6 Question 7

On se place dans control_path et on édite le programme PLA pour ajouter les instructions OR et AND.

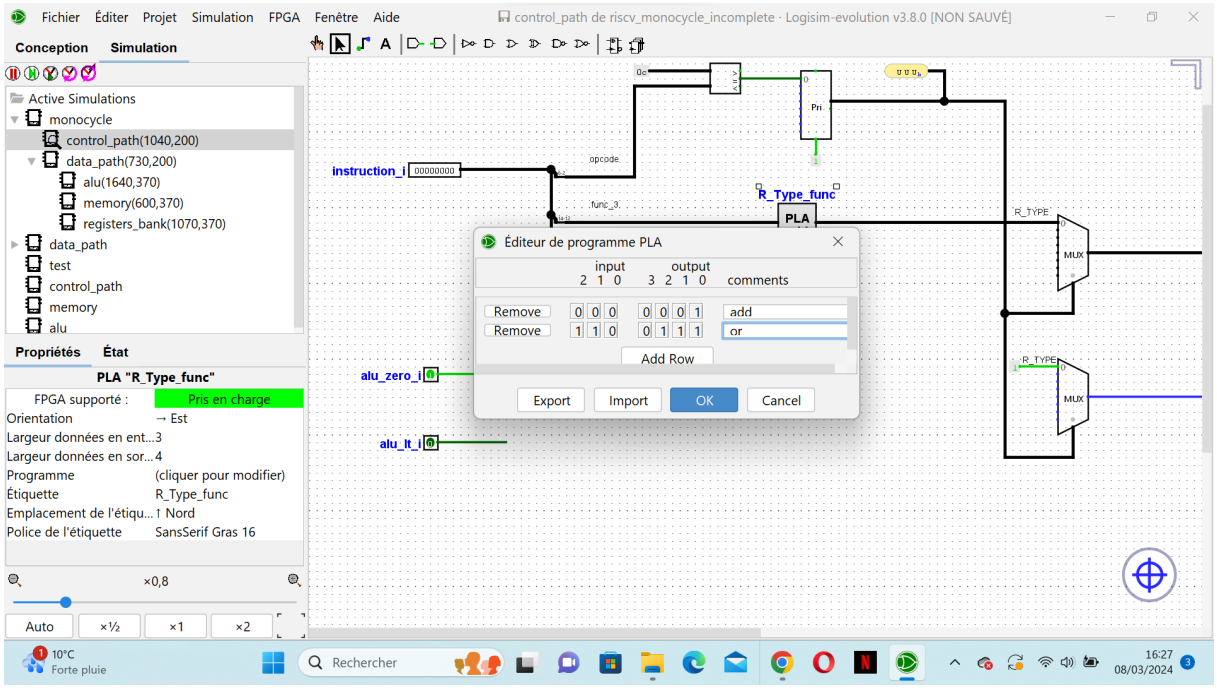


FIGURE 7 – Modofication du Programme PLA pour ajouter les instructions

4 Ajout des types I et U

4.1 Question 8

Pour respécter la spécification RISC-V, il faut utiliser le composant Constante de la librairie câblage et de remplacer le registre nul par la constante nulle en déconnectant le câble qui lui est associé comme ceci :

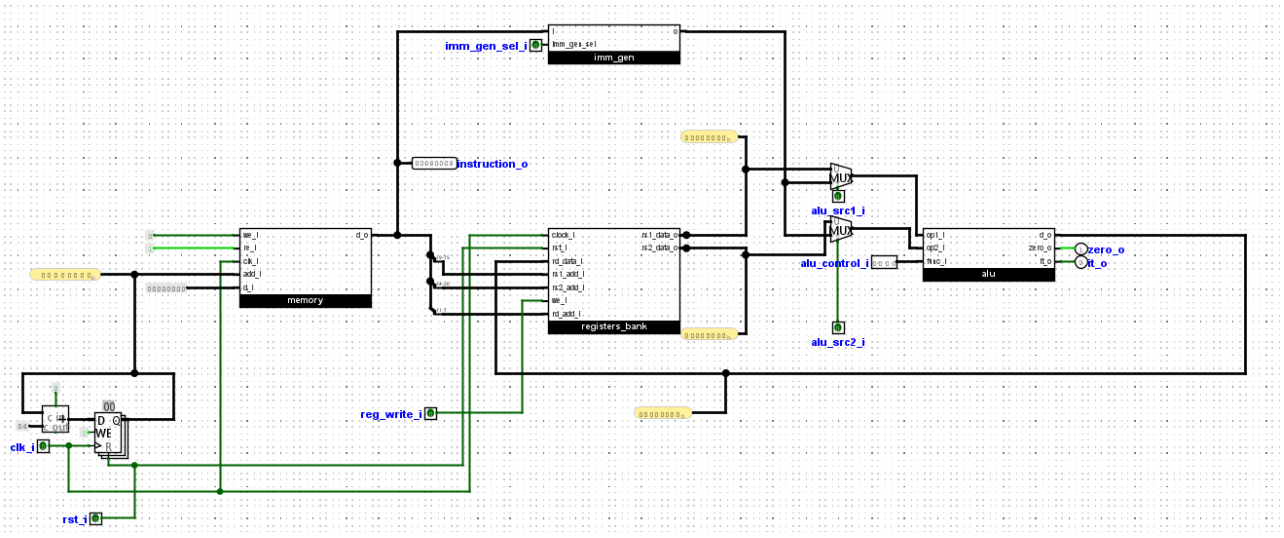


FIGURE 8 – Modification de data_path

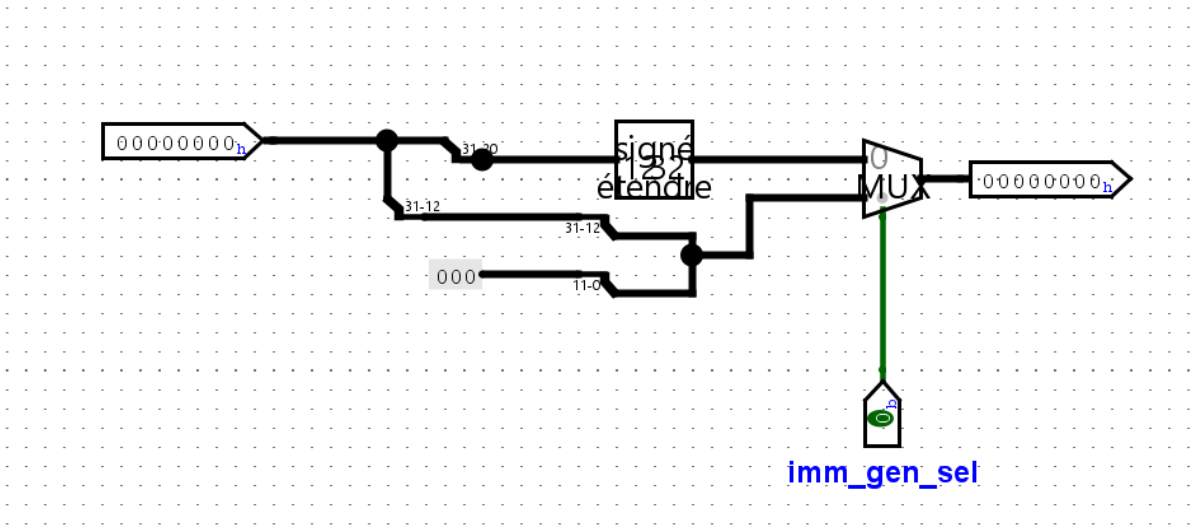


FIGURE 9 – Zoom sur imm_gen_sel

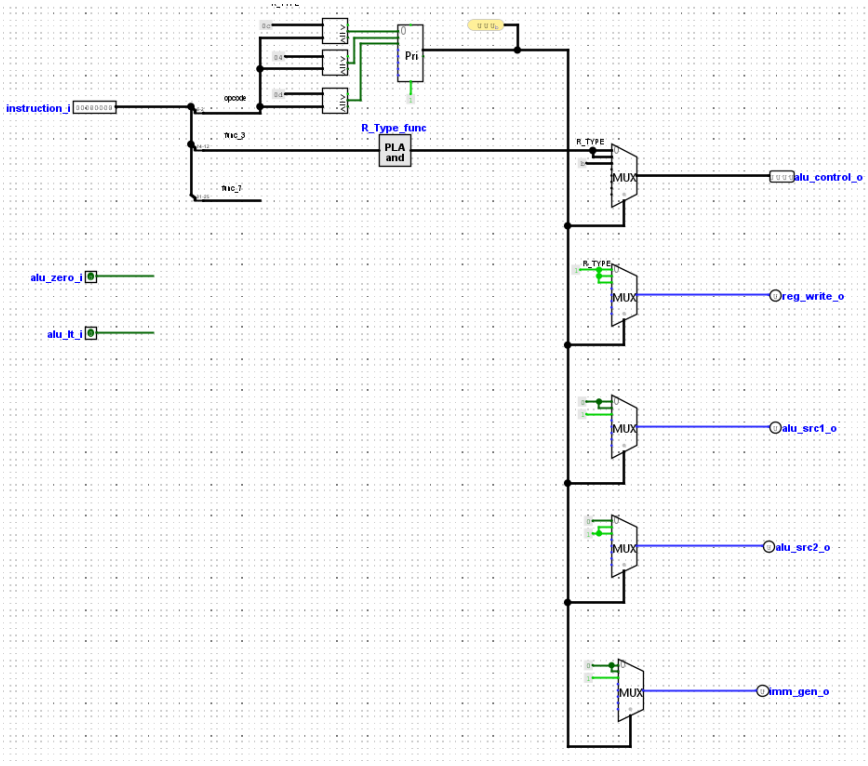


FIGURE 10 – Modification de control_path

4.2 Question 9

0xDEADBEEF contenant 32 bits, on peut assembler le programme en 2 instructions :

$$LUIR1, 0xDEADC = DEADC0B7 \tag{2}$$

$$ADDIR1, 0xEEF = EEF08093 \tag{3}$$

4.3 Question 10

On charge les instructions de la quéstion précédente et en lançant la simulation, on obtient l'implémentation suivante :

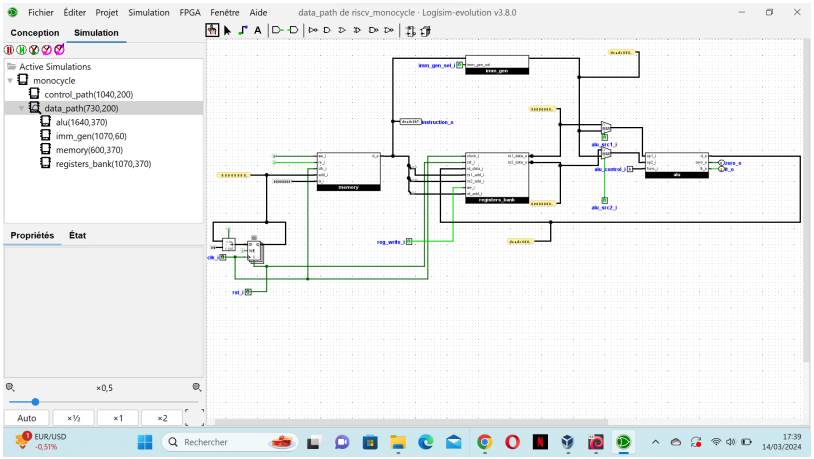


FIGURE 11 – Circuit avec ajout du premier reset

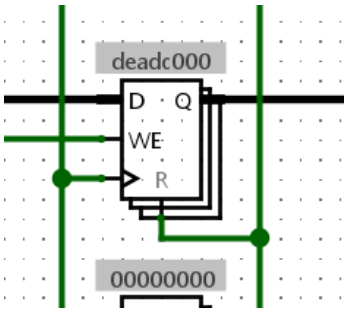


FIGURE 12 – Etat du Registre 1 au premier Clock

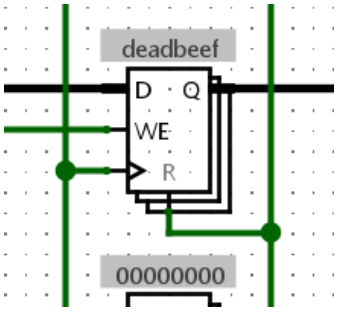


FIGURE 13 – Etat du Registre 1 au second Clock

***

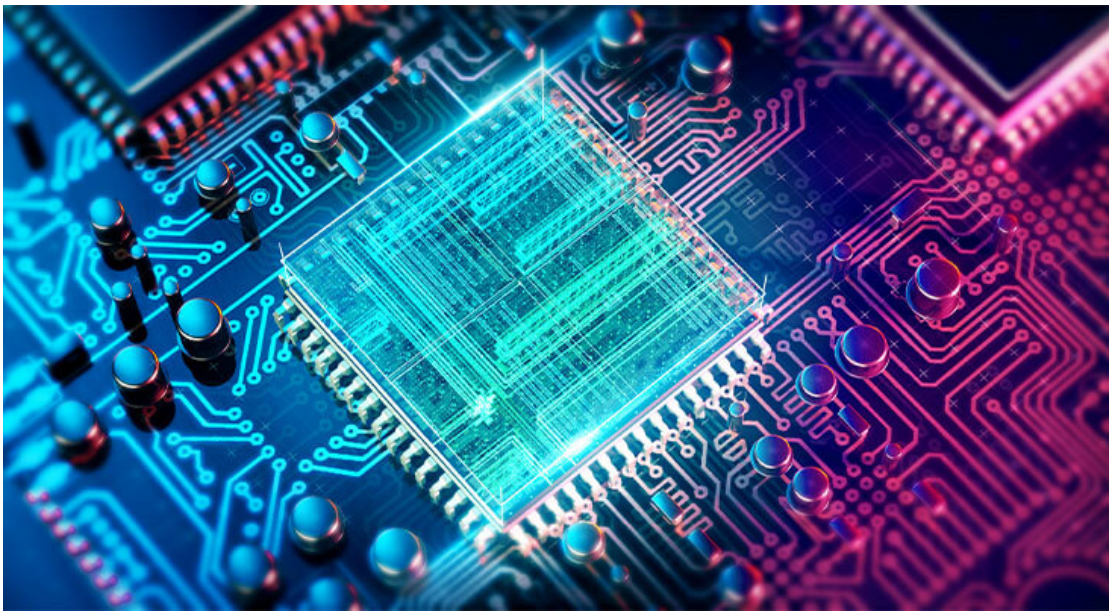


FIGURE 14

FIN DU RAPPORT

Merci pour votre attention !

TALEB Samy, VEYRAT Ylies